Ventajas de JSON-RPC

ESTÁNDAR
Protocolo bien definido
RFC 4627 y JSON-RPC 2.0

INTEROPERABILIDAD
Cualquier lenguaje
puede implementarlo

SIMPLICIDAD
Basado en JSON
Fácil de debuggear

BIDIRECCIONAL
Client→Server
Server→Client

JSON-RPC en el Contexto MCP
Model Context Protocol

JSON-RPC
Remote Procedure Call
usando JSON como formato
de intercambio de mensajes

Conceptos Básicos

FUNCIÓN LOCAL
`let resultado = sumar(5, 3)`

FUNCIÓN REMOTA
`let resultado = llamarRemoto('sumar', [5,3])`

Códigos de Error JSON-RPC Estándar

-32700
Parse Error
JSON inválido

-32600
Invalid Request
Estructura incorrecta

-32601
Method Not Found
Método inexistente

-32602
Invalid Params
Parámetros incorrectos

-32603
Internal Error
Error del servidor

Estructura de Mensajes JSON-RPC

REQUEST (Solicitud)

```
jsonrpc: 2.0
method: write_file
params:
  path: archivo.txt
  content: Hola mundo
id: 1
```

RESPONSE SUCCESS

```
jsonrpc: 2.0
result: File written successfully
id: 1
```

RESPONSE ERROR

```
jsonrpc: 2.0
error:
  code: -32602
  message: Invalid params
id: 1
```

Ejemplo Práctico: Escribir Archivo

MENSAJE REQUEST

```
jsonrpc: 2.0
method: filesystem/write_file
params:
  path: C:/temp/test.txt
  content: Hello World
id: 42
```

MENSAJE RESPONSE

```
jsonrpc: 2.0
result:
  success: true
  bytes_written: 11
id: 42
```

DEBUGGING JSON-RPC
- Logs de mensajes JSON
- Inspección de requests/responses
- Timeouts y reconexiones
- Validación de esquemas

Flujo de Comunicación Detallado

1. SOLICITUD CLAUDE
```
filesystem:write_file(
  path='archivo.txt',
  content='datos'
)
```

2. TRADUCCIÓN DESKTOP
Convierte a JSON-RPC
Envía via stdin

3. RECEPCIÓN SERVIDOR
Parsea JSON-RPC
Extrae método y parámetros

4. EJECUCIÓN
`fs.writeFileSync(path, content)`
Operación real en disco

5. RESPUESTA SERVIDOR
Crea JSON-RPC response
Envía via stdout

6. RETORNO DESKTOP
Parsea respuesta
Devuelve resultado

7. CONFIRMACIÓN CLAUDE
Recibe resultado
Continúa conversación

COMANDO DE ARRANQUE
```
npx -y @modelcontextprotocol/server-filesystem
```

1. NPX descarga el paquete
2. Lanza proceso servidor
3. Establece comunicación stdio

Entidades en la Arquitectura MCP

CLAUDE DESKTOP
(Tu Máquina)
Traductor JSON-RPC
Maneja comunicación

SISTEMA OPERATIVO
Windows File System
Acceso real a disco

MCP SERVER
(Proceso Node.js)
Implementa herramientas
Ejecuta operaciones

CLAUDE AI
(Anthropic Servers)
Genera solicitudes
de herramientas MCP

CICLO DE VIDA DEL SERVIDOR MCP
1. Claude Desktop lee configuración
2. Ejecuta comando npx
3. NPX descarga paquete si es necesario
4. Lanza servidor como proceso hijo
5. Establece comunicación JSON-RPC
6. Servidor queda escuchando requests
7. Al cerrar Claude Desktop, mata proceso

Mecanismos de Transporte

STDIO
stdin/stdout
Procesos locales

TCP SOCKETS
Red local/remota
Puertos específicos

WEBSOCKETS
Comunicación web
Tiempo real

NAMED PIPES
Windows/Unix
IPC local